

OPTIMIZATION OF CHACHA20-POLY1305/2133

Flurin Baumann, Michela Bontan, Ioannis Tsagkaropoulos, Jasmine Ziwalig

Department of Computer Science
ETH Zurich, Switzerland

ABSTRACT

This project presents the implementation and performance optimization of the Poly2133 and ChaCha20-Poly1305/Poly2133 Authenticated Encryption with Associated Data (AEAD) scheme. We evaluated both scalar and AVX2 SIMD implementations in C on an Intel Core i7-7700HQ processor. Our best ChaCha20 encryption variant achieved 40% of the vectorized peak, while it outperforms OpenSSL for plaintexts < 8 KB, maintaining 80% of its throughput for larger inputs. Poly1305 authenticator has a $10\times$ speedup over the baseline and achieves 50% of the theoretical SIMD peak performance. Scalar optimizations of Poly2133 approached the scalar peak, but high register pressure limited vectorized throughput to 30% of its theoretical maximum. Our most performant version of ChaCha20-Poly1305 reached a peak of 9.92 ops/cycle and achieved almost 73% of OpenSSL's performance.

1. INTRODUCTION

Motivation. Secure communication is a fundamental right of internet users. In practice, the confidentiality and integrity of transmitted data are ensured by AEAD schemes. Among these, ChaCha20-Poly1305 is widely deployed in TLS and QUIC and is the focus of this paper. Unforgeability of authentication tags is also critical, therefore we also optimized a more secure extension of Poly1305, the Poly2133. Efficient implementations of AEAD schemes directly impact network throughput and user experience amid growing volumes of real-time encrypted traffic.

Achieving high performance is far from trivial. ChaCha20 stream cipher contains sequential dependency chains, while SIMD vectorization typically requires processing multiple plaintext blocks in parallel, leading to significant register pressure. Poly1305's large-integer arithmetic is typically implemented using smaller limbs, requiring the management of numerous intermediate products and carry propagations. These addition-with-carry chains restrict instruction-level parallelism (ILP) and can stall the processor pipeline. The state-of-the-art library OpenSSL has optimized implementations of ChaCha20-Poly1305 and serves as our benchmark for evaluating our implementation. OpenSSL uses

hand-written assembly implementations that exploit SIMD instructions, dynamically selecting the fastest code path supported by the target CPU. For Poly2133, we tweaked the given python script to support the target prime field to generate test cases for our C implementations.

Contribution.

We aim to implement an optimized vectorized C implementation of the ChaCha20-Poly1305, extend the Poly1305 to prime field $2^{213} - 3$ and compare our benchmarks with OpenSSL.

2. BACKGROUND ON THE ALGORITHM/APPLICATION

This section presents the theoretical background on ChaCha20-Poly1305, Poly2133, the cost measure and cost analysis we later use to create performance plots.

ChaCha20.

The ChaCha20 block function generates a 64-byte pseudorandom keystream. It operates on sixteen 32-bit words initialized with 4 constants, a 256-bit key, a 32-bit block counter, and a 96-bit nonce. Eighty Quarter-Rounds (QR), each consisting of 12 sequential addition, rotation, XOR operations on 4 state elements, are executed and the final state is added to the initial state to yield the keystream [1].

The ChaCha20 stream cipher encrypts data in 64-byte blocks by XORing each plaintext block with the corresponding keystream block generated by the ChaCha20 block function using the appropriate block counter.

Poly1305/2133.

Poly1305 and Poly2133 are one-time Message Authentication Codes (MAC) that take a 32/54-byte key and an arbitrary-length message and produce a 16/26-byte authentication tag. The key is split into two 16/26-byte values, r and s , interpreted as little-endian integers. The r is clamped by clearing selected high and low bits. The message is processed in 16/26-byte blocks, each interpreted as a little-endian integer. Using a zero-initialized accumulator, each block is processed as $acc = ((acc + block) * r) \bmod p$, using prime field $p = 2^{130} - 5$ for Poly1305 and $p = 2^{213} - 3$ for Poly2133. After all blocks are processed, s is added to the accumulator and the 128/208 least significant bits are

serialized in little-endian order to form the tag. Poly2133 provides stronger unforgeability security than Poly1305 as security bounds are tied to the tag's size [2].

Chacha20-Poly1305/2133.

The ChaCha20-Poly1305/2133 combines the ChaCha20 stream cipher for encryption with the Poly1305/2133 MAC for integrity. ChaCha20-Poly1305/2133 derives a one-time Poly1305/2133 key from the ChaCha20 block function (32/54-bytes), encrypts the plaintext with ChaCha20 and authenticates the mac data with Poly1305/2133. The AEAD output is the resulting tag and ciphertext [3].

```
aead_encrypt(aad, key, nonce, ptxt):
    poly_key = chacha_block(key, 0, nonce)
    ctxt = chacha20_encrypt(key, 1, nonce, ptxt)

    aad = aad + pad16(aad)
    ctxt = ctxt + pad16(ctxt)
    aad_l = num_to_4_le_bytes(aad.length)
    ct_l = num_to_4_le_bytes(ctxt.length)
    mac_data = aad | ctxt | aad_l | ct_l

    tag = poly1305/2133_mac(mac_data, poly_key)
    return (ciphertext, tag)
```

Cost measure.

Our cost measure is the number of all integer operations, loop increments, array index computations and comparisons. A thorough analysis of operation count parametrized by input length can be found in our repository team33 ({function}/op-computation.py). We were unable to directly measure integer operations in the OpenSSL implementation, since the processor used in our experiments did not support hardware integer events. We also tried cachegrind [4], but this included all executed instructions rather than integer operations alone. For comparison purposes, we consequently assumed the same operation count as our best vectorized implementation. Here we specify an indicative cost analysis for our baseline implementations. Let L_p, L_a denote the length of the plaintext and Additional Authenticated Data (AAD) in bytes.

ChaCha20 Cost. The core of the ChaCha20 encryption algorithm processes data in blocks of 64 bytes. For each block, the algorithm executes 80 QR totaling 1920 iops (rotation is decomposed into 2 shifts, 1 SUB and 1 OR operation). Accounting for data initialization, loop and state increments, array index computations and ciphertext computations, the total cost for ChaCha20 encryption is:

$$W_{\text{chacha}}(L_p) = 2 + 1983 \cdot \left\lfloor \frac{L_p}{64} \right\rfloor \text{ iops}$$

Poly1305 Cost. The initialization phase of Poly1305 prepares the internal keys, clamps r and calculates number of full blocks, totaling 24 operations. Let L_m be the size of padded data on which tag creation operates on. These are

processed in 16-byte blocks. For each block, the implementation incurs 9 iops and calls to limb conversion (`to_large_num_rep`), limb-wise addition (`add_large_nums_55`), and multiplication modulo p (`mulmod_p`). The collective cost of these 3 functions is $C_{\text{block}} = 139 + 27 + 114 = 280$. Finalizing the tag requires addition with s (`add_large_nums_54`) and extraction of the 16 least significant bytes of the result (`to_16_le_bytes`), with a cost of $C_{\text{tail}} = 48 + 60 = 108$. Thus, the tag creation cost is:

$$W_{\text{tag}}^{1305}(L_m) = \left\lfloor \frac{L_m}{16} \right\rfloor \cdot (9 + C_{\text{block}}) + C_{\text{tail}} \text{ iops}$$

Poly2133 Cost. For Poly2133 we use $18 + 2 \cdot 268 = 554$ operations to prepare the internal keys r and s , clamp r and convert both to 8-limb format. The tag creation follows the same structure as Poly1305, but processes data in blocks of 26-bytes. Only the constants C_{block} and C_{tail} change, which now entail the cost of the corresponding functions of Poly2133. Specifically, `to_large_num_rep`, `add_large_nums_88` and `mulmod_p` which operate on 8-limb numbers make up the cost of $C_{\text{block}}^{2133} = 860$ ops. The functions extracting the 26 least significant bytes for the tag (`add_large_nums_88`, `to_26_le_bytes`) have a cost of $C_{\text{tail}}^{2133} = 319$. Thus, the tag cost is:

$$W_{\text{tag}}^{2133}(L_m) = 3 + \left\lfloor \frac{L_m}{26} \right\rfloor \cdot (9 + C_{\text{block}}^{2133}) + C_{\text{tail}}^{2133} \text{ iops}$$

ChaCha20-Poly1305 Cost. Combining the ChaCha20 and Poly1305, the total operation count W_{total} for encrypting and authenticating a plaintext and AAD is:

$$W_{\text{total}}(L_p, L_a) = 24 + W_{\text{chacha}}(L_p) + 22 + W_{\text{tag}}^{1305}(L_p + L_a + 2).$$

Asymptotically, the total operational complexity scales linearly with the input sizes L_p, L_a . Under the assumption that the plaintext significantly dominates the AAD ($L_p \gg L_a$), the overall cost simplifies asymptotically to $\mathcal{O}(L_p)$.

ChaCha20-Poly2133 Cost. Similarly, The total operation count of the combined ChaCha20-Poly2133 is:

$$W_{\text{total}}(L_p, L_a) = 24 + W_{\text{chacha}}(L_p) + 554 + W_{\text{tag}}^{2133}(L_p + L_a + 2),$$

where the asymptotic behaviour of the operational complexity is identical to ChaCha20-Poly1305.

3. OPTIMIZATION PERFORMED

This section presents our optimizations for algorithms presented in section 2 guided by profiling insights and microarchitectural restrictions. For all optimizations, we assume the target platform is little-endian which allows casting byte arrays to `uint64_t` pointers, eliminating unnecessary byte-wise operations, such as replacing byte-wise ciphertext computation with 64-bit XOR operations.

ChaCha20 Stream Cipher.

The baseline implementation initializes the internal state and processes the plaintext sequentially in 64-byte blocks. For each block, the ChaCha20 block function generates a 64-byte keystream which is XORed with the plaintext to produce the ciphertext. Execution profiling [5] showed that keystream generation dominates the computation time.

Firstly, we perform strength reduction on 1 integer division and 1 modulo operation for block calculations. Next, we inline the state initialization and ChaCha20 block functions to avoid function call overhead. Then, we unroll the final state-addition loop by 4 and use ILP to perform independent state additions concurrently (bounded by the number of available scalar ALU ports). At this point, the working state is read before and written after every double round. Since the working ChaCha20 state contains only sixteen 32-bit words, it can reside entirely in registers. Thus, next on we apply scalar replacement, loading the state into scalar variables at the beginning of block processing and we write it back only after the final keystream has been produced. This removes redundant memory traffic and improves temporal locality. Fully unrolling the ciphertext computation loop exposes eight independent 64-bit XOR operations, enabling the compiler to schedule them concurrently and further increase ILP. Subsequently, we investigate inter-block parallelism as different plaintext blocks are handled independently from each other. We unroll the outer loop by 4 and use ILP to process 4 plaintext blocks simultaneously. This requires 4 independent ChaCha20 states, 10 double rounds per state, and 4 ciphertext computations in parallel. However, this optimization produces negligible performance gains, consistent with the processor’s microarchitecture. Specifically, the optimization before that has already saturated the available scalar ALU execution ports, leaving little opportunity for additional throughput.

Since a ChaCha20 state consists of sixteen 32-bit words and a 256-bit YMM register can hold 8 such words, we vectorize the algorithm by processing 8 blocks simultaneously using 16 vectors. Vector i contains the i -th state word from 8 independent chacha states. Our vectorized QR implementation exploits this to compute the QRs of all 8 blocks at once. Moreover, our first vectorized implementation constructs the state vectors using 16 `_mm256_set_epi32` and 8 `memcpy` calls for every loop iteration and extracts the resulting keystream using 128 `_mm256_extract_epi32` calls. While this successfully vectorizes the ChaCha20 computation itself, the frequent packing and unpacking of vectors introduces substantial overhead. Moreover, ciphertext computation remains scalar because the keystreams are extracted into arrays. Then, we eliminate the unnecessary data rearrangement and remove scalar extraction overhead. State vectors require only 16 `_mm256_set_epi32` calls for all the function lifetime, and the generated keystream remains

in vector form throughout the computation. Two vectorized 8×8 matrix transpositions produce vectors holding keystreams, enabling vectorized plaintext loads, XOR operations and ciphertext stores. Finally, since AVX2 does not include a vectorized rotation we replace our engineered 3-instruction rotation with a `_mm256_shuffle_epi8` using precomputed shuffle masks for rotations by 8 or 16 bits. This reduces the operation count of the vectorized QR by 20% and produces up to a 12% decrease in runtime.

Poly1305. We represent the 130-bit evaluation key r with 32-bit unsaturated, balanced limbs. Execution profiling of the baseline `create_tag` revealed that multiplications in the prime field (`mulmod_p`) and conversion of data blocks to limb representation (`to_large_num_rep`) account for 37% and 36% of the computation respectively. Firstly, we reduce the op count and get rid of the `for` loop in `to_large_num_rep`, breaking sequential dependency chains and exposing ILP. Secondly, we inline the critical field operations within `create_tag` to eliminate function call overhead. Next, we unroll the data handling loop by 4 and introduce ILP to process 4 message blocks simultaneously. We precompute up to the 4th power of r to allow each block to be processed by an independent accumulator variable via the recurrence relation $acc_i \leftarrow acc_i \cdot r^4 + M_i$ for $i \in \{1, 2, 3, 4\}$ [6]. Upon loop termination, the 4 accumulators are realigned by multiplying them by descending powers of r before being aggregated via $acc \leftarrow acc_1 \cdot r^4 + acc_2 \cdot r^3 + acc_3 \cdot r^2 + acc_4 \cdot r$. This design increases register pressure, as 4 distinct powers of r must reside continuously in the registers. Still, this optimization yielded a $2\times$ speedup over the baseline. Subsequently, we implemented a 2-carry-delay by loading 8 message blocks at a time and deferring the expensive carry propagation and modular reduction once every 2 blocks. This optimization performs $acc_i \leftarrow acc_i \cdot r^8 + M_i \cdot r^4 + M_{i+4}$ for $i \in \{1, 2, 3, 4\}$ followed by a unified carry-reduction pass. Although this optimization introduces the minor overhead of storing an additional constant r^8 , it unrolls the message blocks loop by 8 and add ILP to maximize instruction independence. This reduces the total instruction count by 30% relative to the parallel Horner implementation and decreases memory writes by 66%, since the compiler stores intermediate limbs in registers for longer execution stretches. This reaches a $4\times$ speedup over the baseline. We precompute the prime-field limbs $5 \cdot r^4$ and $5 \cdot r^8$, as these modular reduction operations occur frequently during field multiplication. Profiling shows a minor decrease in total instruction count ($< 3\%$) and a notable reduction in internal intermediate data-shuffling/alu operations inside the execution block (18%). However, this precomputation does not yield a noticeable performance improvement, since multiplications by small constants (like $*5$) are optimized away during compiled with `-O3` flag.

The vectorized optimization preserves the logic of the

carry-delay implementation, differing only in data representation and memory layout. In the scalar baseline, accumulator limbs are stored as `uint32_t` values, casted to `uint64_t` for intermediate multiplications, and reduced back to `uint32_t` after carry propagation. The vectorized design eliminates this casting overhead by loading operands directly into 64-bit SIMD lanes, maintaining a consistent data type throughout the computation. Specifically, each SIMD vector packs 4 independent 64-bit variables. Since the unsaturated representation of our field elements spans 5 limbs, each variable is structurally mapped as an array of 5 SIMD elements, where each element isolates a distinct limb across 4 parallel message blocks. This data alignment mirrors the 4 parallel accumulators utilized in our scalar design, mapping them natively onto the 4 vector lanes without leaving any unused bits or computational capacity inside the SIMD registers.

Poly2133. For the implementation of Poly2133 we opt for using an unsaturated, unbalanced representation of 8×32 -bit limbs to accommodate the larger prime $p = 2^{213} - 3$. We choose 32-bit limbs instead of 64-bit even though they come with double the carry propagations so we can avoid `uint128_t` during the multiplication loop, which would be hindering vectorization later. Specifically, $7 \times 28 + 17$ -bits are used to represent large numbers, leaving 4-bits of headroom for delayed carry. The message is processed in 26-byte blocks since this is the largest possible size still within the prime field to reduce `mulmod_p` calls. Profiling with `callgrind` [5] showed that 67% of the runtime is spent within the `mulmod_p` function, which multiplies the accumulator with r modulo p , while `cachegrind` results [4] confirm that the working set stays within cache since it's only around 224 bytes. Furthermore, since the implementation is compute bound, we focus our optimizations on the computations of `mulmod_p` rather than cache.

For scalar optimizations, we first inline all functions to eliminate function call overhead and then unroll all loops independent of carries in order to add in ILP, and also do small precomputations and reduce the amount of memory accesses. Next, we break the serial dependency chain of the polynomial evaluation and implement the 2-level approach [6] with 4 blocks, where in each iteration we multiply each block as well as the overall accumulator with their respective precomputed power of r and then add them all together. Instead of doing 5 multiplications, modulus, repackings and carry propagations per 4 blocks, we delay this and do them only once every 4 blocks. This is possible because the results of all multiplications of the blocks and the accumulator are smaller than 64-bits, which is the limb size of the arrays we use to store the multiplication results before repacking them back into the accumulator representation of $7 \times 28 + 17$ -bits (in the worst case, each block and the accumulator can add up to 7 multiplications

to one limb and get multiplied by the wraparound factor 3, so: $7 \cdot 3 \cdot 5 \cdot 2^{28} \cdot 2^{28} < 2^{64}$). This way we delay repacking, carry propagation and modulo, doing them only once every 4 blocks, as well as fusing 5 multiplication loops into one. Finally, we reduce the number of live variables the compiler has to keep in registers by fully unrolling and restructuring the multiplication loop, so that we now add up all multiplications for a specific limb, then instantly store the result along with the prior carry in the corresponding accumulator limb and extract the carry for the next limb to move on. Furthermore, we replace the block limbs with scalars giving the compiler full freedom in register allocation and we can now use two `uint64_t` variables instead of two `uint64_t` arrays to store the multiplication results, further reducing register pressure. While this approach creates a serial addition chain when summing limb products, the reduction in register pressure makes it a worthwhile compromise. Furthermore, addressing this pressure is helpful for effective vectorization.

For vectorization we load specific limbs of blocks in a temporary vector using `_mm256_set_epi64x`, essentially splitting the serial addition chain into 4 parallel accumulators in the SIMD lanes, multiplying each with the corresponding vector holding powers of r and reducing the 4 results into 1 after each limb. The amount of parallel blocks we use now introduces a trade-off between how many reductions and carry propagations we have to do per block and how many scalar variables we introduce. In total there are 16 reductions (2 for each limb). Going up to 12 blocks would introduce $4 \times 8 = 32$ new variables the compiler has to juggle but reduces the number of reductions per block only by $16/8 - 16/12 = 2 - 1.3 = 0.7$, while going from 4 to 8 blocks reduces them by $16/4 - 16/8 = 4 - 2 = 2$ and only introduces 32 new variables as well, which demonstrates the diminishing returns. This is one reason we stop at 8 blocks. Another reason is that the 64-bit multiplication accumulators in our delayed carry implementation would overflow for 12 blocks ($7 \cdot 3 \cdot 13 \cdot 2^{28} \cdot 2^{28} > 2^{64}$).

ChaCha20-Poly1305/Poly2133. For the benchmark of the AEAD scheme, we create 3 variants. Since the AEAD scheme uses the ChaCha20 encrypt, Poly1305/2133 init and Poly1305/2133 create tag function, we first create a baseline implementation of the AEAD scheme using the baseline implementations of its subsequent parts. Then we create the best scalar and vectorized variant, using the best scalar and vectorized implementations of its parts respectively.

4. EXPERIMENTAL RESULTS

This section gives the hardware and software specifications for the machine used to run the algorithms of 2 as well as experimental results of the optimizations described in 3.

Experimental setup. All experiments were conducted on an Intel Core i7-7700HQ processor, with Kaby Lake microarchitecture, with 4 scalar ALU ports and 3 256-bit SIMD ALU ports. The processor supports AVX2 vector instructions. The cache hierarchy consists of a 32KB L1 data cache and a 256KB L2 cache per core, as well as a shared 6MB L3 cache. The benchmarks were compiled using GCC 13.3.0. Scalar variants were compiled with `-O3 -fno-tree-vectorize -march=native`, while vectorized ones used `-O3`. Runtime was counted using the infrastructure of the assignments, under cold caches. All data were 32-byte aligned. We tested plaintext sizes starting from 1 KiB, progressing in powers of 2, up to at least 32 MiB ($> 4\times$ the L3 cache size).

Results (ChaCha20 Stream Cipher). The scalar baseline achieves roughly 2.2 ops/cycle as seen in Fig. 1. After applying strength reduction and inlining, performance improves to about 2.7 ops/cycle. Inlining the QRs via macros increases compiler visibility into the computation. This results in more effective instruction scheduling and reordering, increasing the ILP and allowing better utilization of execution ports when independent QR instances are available. However, this variant entails a loop-carried dependency in the ciphertext computation. Unrolling this loop fully and using ILP removes this bottleneck, improving utilization of the available computational units. Our final scalar optimization, i.e. inter-block parallelism by unrolling the loop handling plaintext blocks by 4 and using ILP, proved worse. While this increases the number of independent operations available to the scheduler, maintaining 4 ChaCha state instances (64 32-bit numbers) exceeds the availability of architectural registers, leading to register spilling. This introduces load/store dependencies, which prevent temporal locality and compete with integer computations. Our best scalar variant (`unroll ilp ctxt`) achieves 70.5% of scalar peak.

The vectorized baseline starts at roughly 8 ops/cycle. Our SIMD variants outperform OpenSSL for small inputs (< 8 KB), since OpenSSL prioritizes sustained throughput over minimizing initialization and scheduling overhead. From Fig. 2 we observe that the best runtime is attained by `vectorized3`, where we replace the 3-instruction rotation with a `_mm256_shuffle_epi8` for rotations by 8 or 16 bits. This reduces op count more than 30% compared to the first vectorization, thus we did not include it in the performance plot. For the record, its performance is worse than `vectorized2`'s. Therefore, the variant with the best performance is (`vectorized2`), reaching 9.57 ops/cycle at 8 KB, 80% of OpenSSL's performance for large inputs (> 6 MB), almost 40% of the vectorized peak and a overall $4.1\times$ speedup from the baseline implementation. This is still far from the theoretical SIMD peak of 24 ops/cycle on a machine with three AVX2-capable vector ALU ports. We argue this gap is fundamental to our implementation

Performance of ChaCha20 Encrypt

iops/cycle

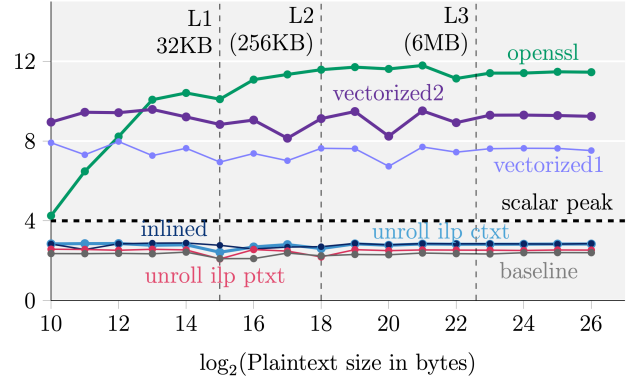


Fig. 1. Performance of ChaCha20 Encrypt. Scalar variants are compiled with `-O3 -fno-tree-vectorize` and have approximately the same operation count. Vectorized variants are compiled with `-O3`. "vectorized1" has approximately 15% more iops than the other vectorized variants. Vectorized variants do not have comparable op count with scalar ones.

and cost measure. The vectorized peak assumes all vector ALU ports are fully pipelined, but for our implementation this does not hold. Firstly, our cost measure does not capture the `16 _mm256_permute2x128_si256` and the `32 _mm256_unpack{lo,hi}_epi{32,64}` instructions which happen once every 8 plaintext blocks. Secondly, during the unpack-and-permute keystream extraction, intermediate results must be materialized and reshuffled, limiting ILP. Moreover, when processing 8 plaintext blocks concurrently, the 16 YMM registers are fully occupied by state vectors and the keystream extraction pipeline. As a result, plaintext loads and ciphertext stores cannot be kept in registers and begin to spill into memory. These spill/reload operations introduce additional latency and reduce the number of independent ALU instructions that can be processed.

Finally, we observe pronounced performance drops in the vectorized implementation, for plaintext sizes of 128 KiB, 4 MiB. The algorithm's working set (working state, keystream processing buffers for the current 64-byte block) resides entirely in L1 cache. However, as plaintext size grows beyond 128 KiB and exceeds L1 capacity, memory accesses spill into L2 cache. At 4 MiB, the plaintext and ciphertext buffer sizes exceed L3 cache, forcing frequent DRAM accesses with high cycle latencies. When memory latency becomes the limiting factor, the CPU can only hide this latency by executing independent instructions in parallel while waiting for data to return. Our scalar implementations have more independent instruction to execute compared to the vectorized code, which enables better masking

Runtime Speedup of ChaCha20 Encrypt Variants

Compiler Flags: -O3 march=native

Speedup (wrt. cycles)

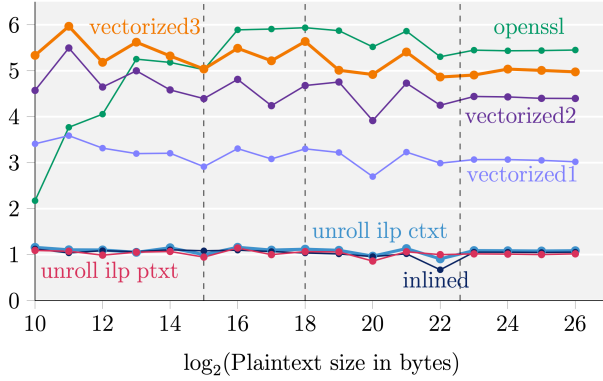


Fig. 2. Runtime speedup of ChaCha20 encrypt variants with respect to the baseline implementation.

of the memory latency for the former.

Poly1305. The benchmarking results in Fig. 3 show that inlining achieves only marginal gains over the baseline, with a performance of 1 ops/cycle. The major performance improvements come from reducing dependency chains and increasing ILP. Assuming the host machine is little-endian, we eliminate the sequential loop in `tolarge_num_rep` exposing parallelism in limb construction. Loop unrolling and ILP in the multiple accumulator implementation (parallel Horner) improve throughput by allowing multiple independent accumulation paths.

Further gains are achieved by the carry-delay, where carry propagation and modular reduction are amortized across multiple blocks. This reduces the frequency of expensive normalization steps and increases instruction throughput. Our best scalar variant (inlined carry delay parallel Horner), reaches 50% of the scalar peak.

The vectorized variant achieves an $8\times$ speedup over the optimal scalar implementation, confirming that the dominant bottlenecks in the scalar code are limited ILP and restricted scalar register width. Additionally, the vectorized pipeline eliminates the 32-to-64-bit type casts required by the scalar version, thereby reducing overhead. Consequently, we observed a 75% reduction in executed CPU instructions, even though the total operation count remained identical. Compared to the baseline, our best vectorized variant yields a $10\times$ speedup, reaching a performance of 12 ops/cycle, which corresponds to exactly half of the theoretical vectorized peak. However, our implementation primarily relies on the `_mm256_add_epi64()` and `_mm256_mul_epu32()` SIMD intrinsics, which forces a mix of 32-bit and 64-bit operations. The theoretical vectorized peak is inherently unattainable for our implementation. Precisely, the peak

is achieved under SIMD instructions that process 8 32-bit operations per cycle, whereas our implementation relies on `_mm256_mul_epu32()`, which performs only 4 multiplications per cycle.

Our SIMD implementation outperforms OpenSSL for small input sizes up to 8 KB, which corresponds to a quarter of the L1 data cache capacity. However, as the input size scales to fill the L2 cache, OpenSSL overtakes our best vectorized variant, achieving 33% higher throughput.

Performance of Poly1305 Create Tag

iops/cycle

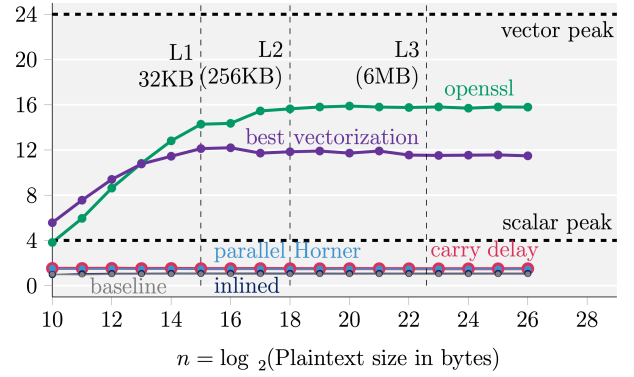


Fig. 3. Performances of Poly1305 optimization variants, compared against OpenSSL.

Poly2133. From the performance plot in Fig. 4 we see that the baseline starts at around 2 ops/cycle, adding inlining and unrolling with ILP, small precomputations and less memory access improves performance slightly. The 2-level approach, which allows for processing multiple blocks in parallel, increases performance to approximately 2.5 ops/cycle. Adding delayed carry on top of it results in a performance of around 3 ops/cycle. Restructuring the multiplication loop and adding scalar replacement results in the best scalar performance, close to the 4 ops/cycle peak. In contrast, precomputing the powers of r which was built on top of delayed carry results in a worse performance only slightly better than the baseline. This is likely due to increased register spillings stemming from the additional variables, which overshadows the computational gain of the saved multiplications. Since this precomputation was still used in later optimizations (scalar replacement and vectorization), it may have negatively contributed to their performances, especially for vectorizations, which already struggle with register pressure.

The vectorized versions for 4 and 8 blocks both plateau at around 7 and 8 ops/cycle with the 8 block version having lower performance for small plaintext sizes and then overtaking after the initial setup cost is amortized. One can see that the gain from 4 to 8 blocks is very small, demon-

Performance of Poly2133 Create Tag

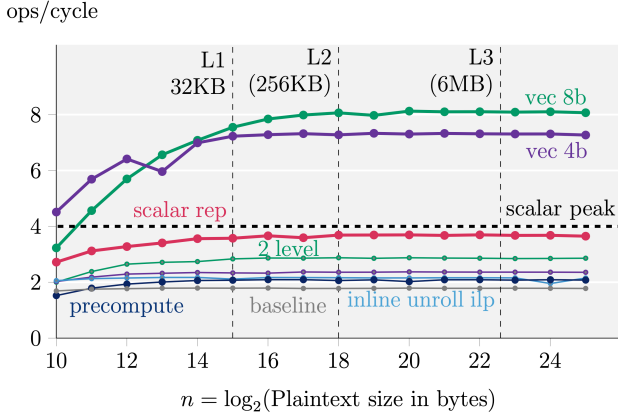


Fig. 4. Performances of Poly2133 optimization variants.

strating the diminishing returns when adding more blocks, which is the reason we stop at 8 blocks. In contrast to Poly1305, the scalar optimizations get close to the scalar peak of 4 ops/cycle; however the vectorized versions only reach around a third of the vector peak. This could be due to optimizations focusing on the `mulmod_p` function resulting in greater improvements for Poly2133 since it is clearly the biggest bottleneck. In contrast, Poly1305’s implementations are less impacted by such optimizations since `to_large_num_rep` is also a significant bottleneck. However, Poly1305 achieves better vectorization results probably because it faces less register pressure, since it only needs to handle 5 limbs instead of 8, which comes with a lot less variables during the multiplication loop.

ChaCha20-Poly1305. The AEAD has a sequential dependency of ChaCha20 and Poly1305, i.e. Poly1305 can only run after ChaCha20 is finished. The baseline implementation reaches a maximum of 1.6 ops/cycle. In Fig. 5 we observe that the scalar version already achieves a $2\times$ runtime speedup over the baseline. Poly1305 is the scalar variant’s bottleneck since its serial carry chain is limiting the scalar optimization potential. The vectorized implementation achieves a throughput of 9 ops/cycle, remaining approximately 30% slower than the OpenSSL implementation. This result highlights how the performance bottleneck shifts between the two primitives. Poly1305 exhibits a block-parallel structure that maps naturally onto SIMD lanes, allowing it to benefit substantially from vectorization. In contrast, while ChaCha20 efficiently exploits scalar execution resources through its independent QR operations, its vectorized implementation requires costly matrix transpositions and data rearrangements, limiting achievable throughput. Given the aforementioned inherent throughput limitations of ChaCha20 and Poly1305, the combined ChaCha20-Poly1305 construction cannot attain the theoretical vector-

ized peak performance. Further improvements would therefore require increasing the throughput of the dominant component, ChaCha20 or Poly1305, depending on which contributes the largest fraction of the total execution time.

In general, our vectorized implementation achieves a maximum runtime speedup of $8.16\times$ over the baseline implementation for plaintext sizes of 65,536 B, as well as a performance improvement up to $6.2\times$ over the baseline. The gap between these two speedup metrics further illustrates the difference in integer operation count between the baseline and vectorized implementations.

Runtime of ChaCha20-Poly1305 Encrypt

cycles $\cdot 10^8$

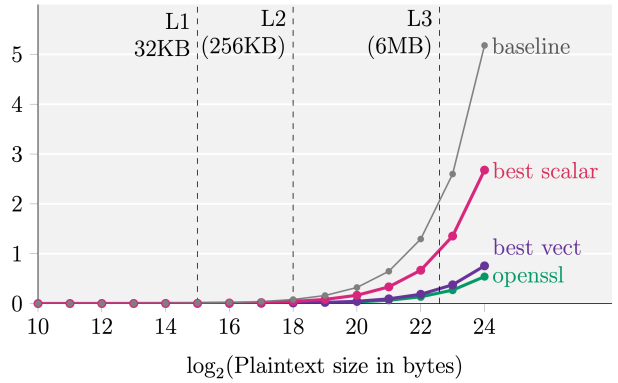


Fig. 5. Runtime of the combined ChaCha20-Poly1305 AEAD scheme, built from the best ChaCha20 and Poly1305 variants, compared against OpenSSL. Operation count between scalar and vectorized variants differs significantly.

5. CONCLUSIONS

We implemented optimized scalar and vectorized versions of ChaCha20, Poly1305, and Poly2133, evaluated them against OpenSSL, and explored the design space of high-performance AEAD implementations. While both implementations achieved notable speedups, Poly1305 vectorized reaching $10\times$ over baseline, the most significant finding is that different cryptographic primitives have fundamentally incompatible optimization strategies. ChaCha20 benefits from scalar execution’s independent QRs but suffers in SIMD from expensive data rearrangement and register pressure. Poly1305 and Poly2133, conversely, struggle with scalar register width and carry-chain dependencies but map naturally to parallel SIMD lanes, achieving greatest speedups when vectorized. This underscores that optimizations demand algorithm specific tuning that respects both computational structure and microarchitectural constraints.

6. REFERENCES

- [1] Daniel J. Bernstein, “ChaCha, a variant of Salsa20,” <https://cr.yp.to/chacha/chacha-20080128.pdf>, January 2008.
- [2] Daniel J. Bernstein, “The Poly1305-AES Message-Authentication Code,” in *Fast Software Encryption (FSE 2005)*, February 2005, <https://cr.yp.to/mac/poly1305-20050329.pdf>.
- [3] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF Protocols,” RFC 8439, RFC Editor, June 2018, <https://datatracker.ietf.org/doc/html/rfc8439>.
- [4] Valgrind Developers, “Cachegrind: a cache and branch-prediction profiler,” [Online]. Available: <https://valgrind.org/docs/manual/cg-manual.html>, 2025, Accessed: Jun. 15, 2026.
- [5] Valgrind Developers, “Callgrind: a call-graph generating cache and branch prediction profiler,” [Online]. Available: <https://valgrind.org/docs/manual/cl-manual.html>, 2025, Accessed: Jun. 15, 2026.
- [6] J. P. Degabriele, J. Gilcher, J. Govinden, and K. G. Paterson, “SoK: Efficient Design and Implementation of Polynomial Hash Functions over Prime Fields,” in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, <https://ieeexplore.ieee.org/document/10646805>.

7. CONTRIBUTIONS OF TEAM MEMBERS

Flurin. Optimizations of chacha encrypt (strength reduction, scalar replacement, unroll and ILP handling multiple plaintext blocks concurrently, first vectorized version), all implementations of Chacha20-Poly2133.

Michela. Optimizations of Poly1305 `create_tag` (inlining, precomputation, parallel Horner, carry delay, vectorization), cost analysis of Poly1305 with Ioannis.

Ioannis. Optimizations of chacha block function (inlining, scalar replacement working state, unroll final state addition loop and use ILP), chacha encrypt (inline, final vectorized versions), all optimizations of Poly1305/2133 key init-clamping, all implementations of Chacha20-Poly1305. Cost analysis ChaCha20, cost analysis of Poly1305 with Michela.

Jasmine. Optimizations of Poly2133 `create_tag` function (limb/implementation choices, inlining, adding ILP, precomputations, 2-level approach, delayed carry, vectorization).

8. CODE

Chacha20 (scalar). Our most performant implementation of the core parts of the ChaCha20 block function, along with the fully unrolled loop which computes the ciphertext, assuming that the host machine is little endian to ensure correctness of casting byte arrays to `uint64_t` *.

```
#define ROTL32(v, n) \
    ( (v << (n)) | (v >> (32 - (n))) )

#define QR(a, b, c, d) \
    a += b; \
    d ^= a; \
    d = ROTL32(d, 16); \
    c += d; \
    b ^= c; \
    b = ROTL32(b, 12); \
    a += b; \
    d ^= a; \
    d = ROTL32(d, 8); \
    c += d; \
    b ^= c; \
    b = ROTL32(b, 7);

uint32_t o0, o1, o2, o3, o4, o5, o6, o7;
uint32_t o8, o9, o10, o11, o12, o13, o14, o15;
for (uint64_t b = 0; b < full_blocks; b++) {
    o0 = st[0]; o1 = st[1]; o2 = st[2];
    o3 = st[3]; o4 = st[4]; o5 = st[5];
    o6 = st[6]; o7 = st[7]; o8 = st[8];
    o9 = st[9]; o10 = st[10]; o11 = st[11];
    o12 = st[12]; o13 = st[13];
    o14 = st[14]; o15 = st[15];

    for (int i = 0; i < DOUBLE_ROUNDS; i++) {
        QR(o0, o4, o8, o12);
        QR(o1, o5, o9, o13);
        QR(o2, o6, o10, o14);
        QR(o3, o7, o11, o15);

        QR(o0, o5, o10, o15);
        QR(o1, o6, o11, o12);
        QR(o2, o7, o8, o13);
        QR(o3, o4, o9, o14);
    }
    ...
    *c_64 = (uint64_t *) (ctxt + idx_start);
    *p_64 = (uint64_t *) (ptxt + idx_start);
    *ks_64 = (uint64_t *) keystream_buffer;

    c_64[0] = p_64[0] ^ ks_64[0];
    c_64[1] = p_64[1] ^ ks_64[1];
    c_64[2] = p_64[2] ^ ks_64[2];
    c_64[3] = p_64[3] ^ ks_64[3];
    c_64[4] = p_64[4] ^ ks_64[4];
    c_64[5] = p_64[5] ^ ks_64[5];
    c_64[6] = p_64[6] ^ ks_64[6];
    c_64[7] = p_64[7] ^ ks_64[7];
}
```

Poly1305 (scalar). Concise version of multiplication in the prime field.

```
r8_5_0 = 5 * r8[0]; r8_5_1 = 5 * r8[1];
r8_5_2 = 5 * r8[2]; r8_5_3 = 5 * r8[3];
```

```
r8_5_4 = 5 * r8[4]; r4_5_0 = 5 * r4[0];
r4_5_1 = 5 * r4[1]; r4_5_2 = 5 * r4[2];
r4_5_3 = 5 * r4[3]; r4_5_4 = 5 * r4[4];

mult0 =
    a[0] * r8[0] + a[1] * r8_5_4 + a[2] * r8_5_3 + \
    a[3] * r8_5_2 + a[4] * r8_5_1 + n0[0] * r4[0] + \
    n0[1] * r4_5_4 + n0[2] * r4_5_3 + n0[3] * r4_5_2 + \
    n0[4] * r4_5_1;

...

mult4 =
    a[0] * r8[4] + a[1] * r8_5_3 + a[2] * r8_5_2 + \
    a[3] * r8_5_1 + a[4] * r8_5_0 + n0[0] * r4[4] + \
    n0[1] * r4_5_3 + n0[2] * r4_5_2 + n0[3] * r4_5_1 + \
    n0[4] * r4_5_0;
```

Poly2133 (scalar). The code below is part of the restructured multiplication in the scalar replacement optimization. We only depict the computation of `mult_small` (result won't overflow $p = 2^{213} - 3$) and `mult_large` (result will overflow p) for the first limb, since the whole code would be too large.

```
// compute mult_small and mult_large for limb 0,
// immediately store in acc[0] and handle carry
// a = block1+acc, b = block2, c = block3, d = block4

ms = a0*(r4)[0] + b0*(r3)[0] +
    c0*(r2)[0] + d0*(r)[0];
ml = a1*three_r4[7] + b1*three_r3[7] +
    c1*three_r2[7] + d1*three_r[7] +
    a2*three_r4[6] + b2*three_r3[6] +
    c2*three_r2[6] + d2*three_r[6] +
    a3*three_r4[5] + b3*three_r3[5] +
    c3*three_r2[5] + d3*three_r[5] +
    a4*three_r4[4] + b4*three_r3[4] +
    c4*three_r2[4] + d4*three_r[4] +
    a5*three_r4[3] + b5*three_r3[3] +
    c5*three_r2[3] + d5*three_r[3] +
    a6*three_r4[2] + b6*three_r3[2] +
    c6*three_r2[2] + d6*three_r[2] +
    a7*three_r4[1] + b7*three_r3[1] +
    c7*three_r2[1] + d7*three_r[1];
sum = ms + ((ml & mask_lowest_17bits) << 11)
+ wrap_carry + carry;
acc[0] = (uint32_t) (sum & mask_lowest_28bits);
carry = sum >> 28; wrap_carry = ml >> 17;

// do for all other limbs too...

// afterwards propagate carry if overflowed
uint64_t total_carry = (carry + wrap_carry) * 3;
uint64_t sum0 = (uint64_t) (acc[0]) + total_carry;
acc[0] = (uint32_t) (sum0 & mask_lowest_28bits);
carry = sum0 >> 28;
acc[1] += (uint32_t) carry;
carry = acc[1] >> 28;
acc[1] &= mask_lowest_28bits;
acc[2] += (uint32_t) carry;
```

Chacha20 (SIMD). Optimized final state computation and keystream extraction (unpack operations perform vectorized matrix transpositions) which allows vectorized computation of ciphertexts. Casts removed for simplification.

```
w0 = _mm256_add_epi32(w0, state0);
// same for w1, w2, w3, w4, w5, w6, w7

l0 = _mm256_unpacklo_epi32(w0, w1);
l1 = _mm256_unpacklo_epi32(w2, w3);
l2 = _mm256_unpacklo_epi32(w4, w5);
l3 = _mm256_unpacklo_epi32(w6, w7);
l_l01 = _mm256_unpacklo_epi64(l0, l1);
l_l23 = _mm256_unpacklo_epi64(l2, l3);
ks0_l = _mm256_permute2x128_si256(
    l_l01, l_l23, 0b00100000);

uint8_t* ptxt_ptr = ptxt + ct_idx;
uint8_t* ctxt_ptr = ctxt + ct_idx;
p0_l = _mm256_load_si256(ptxt_ptr);
c0_l = _mm256_xor_si256(p0_l, ks0_l);
_mm256_store_si256(ctxt_ptr, c0_l);

w8 = _mm256_add_epi32(w8, state8);
// same for w9, w10, w11, w12, w13, w14, w15

__m256i h4 = _mm256_unpackhi_epi32(w8, w9);
__m256i h5 = _mm256_unpackhi_epi32(w10, w11);
__m256i h6 = _mm256_unpackhi_epi32(w12, w13);
__m256i h7 = _mm256_unpackhi_epi32(w14, w15);
__m256i l_h45 = _mm256_unpacklo_epi64(h4, h5);
__m256i l_h67 = _mm256_unpacklo_epi64(h6, h7);
__m256i ks0_h = _mm256_permute2x128_si256(
    l_h45, l_h67, 0b00100000);

__m256i p0_h = _mm256_load_si256(ptxt_ptr+32);
__m256i c0_h = _mm256_xor_si256(p0_h, ks0_h);
_mm256_store_si256(ctxt_ptr+32, c0_h);

//reuse l*, h*, l_l*, l_h*, h_l*, h_h* in different combinations to extract all keystreams
```

Poly1305 (SIMD). Concise form for the multiplication in the prime field (the same structure is used for the remaining 4 limbs of (mult_v))

```
// Limb 0
t0 = _mm256_mul_epu32(a_v[0], r8_v[0]);
t1 = _mm256_mul_epu32(a_v[1], v5);
t2 = _mm256_mul_epu32(t1, r8_v[4]);
t3 = _mm256_mul_epu32(a_v[2], v5);
t4 = _mm256_mul_epu32(t3, r8_v[3]);
t5 = _mm256_mul_epu32(a_v[3], v5);
t6 = _mm256_mul_epu32(t5, r8_v[2]);
t7 = _mm256_mul_epu32(a_v[4], v5);
t8 = _mm256_mul_epu32(t7, r8_v[1]);
t9 = _mm256_mul_epu32(n0_v[0], r4_v[0]);
t10 = _mm256_mul_epu32(n0_v[1], v5);
t11 = _mm256_mul_epu32(t10, r4_v[4]);
t12 = _mm256_mul_epu32(n0_v[2], v5);
t13 = _mm256_mul_epu32(t12, r4_v[3]);
t14 = _mm256_mul_epu32(n0_v[3], v5);
t15 = _mm256_mul_epu32(t14, r4_v[2]);
t16 = _mm256_mul_epu32(n0_v[4], v5);
t17 = _mm256_mul_epu32(t16, r4_v[1]);
```

```
t18 = _mm256_add_epi64(t0, t2);
t19 = _mm256_add_epi64(t4, t6);
t20 = _mm256_add_epi64(t18, t19);
t21 = _mm256_add_epi64(t9, t11);
t22 = _mm256_add_epi64(t13, t15);
t23 = _mm256_add_epi64(t21, t22);
t24 = _mm256_add_epi64(t8, t23);
t25 = _mm256_add_epi64(t20, t24);
mult_v[0] = _mm256_add_epi64(t25, t17);
```

//Limb 1 ... Limb4

Poly2133 (SIMD). The code below is part of the vectorized multiplication with 8 blocks of Poly2133. We use the computation of mult_small of limb 2 to demonstrate how it works since pasting the whole code would not fit the page.

// compute ms, ml for limbs 0 & 1, store results
// and handle carries ...

```
// compute mult_small (ms) for limb 2
vs = _mm256_mul_epu32(
    _mm256_set_epi64x(d0, c0, b0, a0),
    _mm256_set_epi64x(
        (r5)[2], (r6)[2], (r7)[2], (r8)[2]));
vs2 = _mm256_mul_epu32(
    _mm256_set_epi64x(h0, g0, f0, e0),
    _mm256_set_epi64x(
        (r)[2], (r2)[2], (r3)[2], (r4)[2]));
vs = _mm256_add_epi64(vs, vs2);
vs = _mm256_add_epi64(vs, _mm256_add_epi64(
    _mm256_mul_epu32(
        _mm256_set_epi64x(d1, c1, b1, a1),
        _mm256_set_epi64x(
            (r5)[1], (r6)[1], (r7)[1], (r8)[1])),
    _mm256_mul_epu32(
        _mm256_set_epi64x(h1, g1, f1, e1),
        _mm256_set_epi64x(
            (r)[1], (r2)[1], (r3)[1], (r4)[1]))));
vs = _mm256_add_epi64(vs, _mm256_add_epi64(
    _mm256_mul_epu32(
        _mm256_set_epi64x(d2, c2, b2, a2),
        _mm256_set_epi64x(
            (r5)[0], (r6)[0], (r7)[0], (r8)[0])),
    _mm256_mul_epu32(
        _mm256_set_epi64x(h2, g2, f2, e2),
        _mm256_set_epi64x(
            (r)[0], (r2)[0], (r3)[0], (r4)[0]))));
low = _mm256_castsi256_si128(vs);
high = _mm256_extracti128_si256(vs, 1);
```

```
sum_vec = _mm_add_epi64(low, high);
ms = (uint64_t)_mm_extract_epi64(sum_vec, 0)
    + (uint64_t)_mm_extract_epi64(sum_vec, 1);
```

// compute ml for limb 2 ...

```
// add together, store result in acc[2], extract carry
sum = ms + ((ml & mask_lowest_17bits) << 11)
    + wrap_carry + carry;
(acc)[2] = (uint32_t)(sum & mask_lowest_28bits);
carry = sum >> 28;
wrap_carry = ml >> 17;
```

// do for all limbs, then propagate carry
// if overflowed (same as scalar version)